

Distributed Vehicle Network with Decision Aggregation for Dynamic Capacitated Vehicle Routing Problem

Yang Wang, Ya-Hui Jia, *Member, IEEE*, Qiang Yang, *Member, IEEE*, Feng-Feng Wei, *Member, IEEE*, Zhenhong Lin, *Senior Member, IEEE*, Wei-Neng Chen, *Senior Member, IEEE*

Abstract—Dynamic Capacitated Vehicle Routing Problem (DCVRP) is a key challenge in logistics and delivery, in which customer orders arrive dynamically in real-time, and vehicles with capacity constraints need to collaborate to complete delivery tasks. In this paper, we propose a Distributed Vehicle Network with Decision Aggregation (DVNDA) to solve DCVRP. Specifically, we construct a time-driven dynamic problem environment that closely resembles reality, considering real-time customer disclosure. Unlike existing methods that rely on expert knowledge and centralized networks for vehicle allocation, DVNDA allows each vehicle to make independent decisions based on the shared status of all vehicles and customers. In each step, DVNDA synthesizes all vehicles' decisions and selects one to serve the customer, achieving collaboration among the vehicles. Extensive experimental results show that the proposed method significantly outperforms the greedy algorithm and learning-based methods on synthetic and real-world instances of different scales and degrees of dynamics. Our code is available at <https://github.com/ftwangyang/DCVRP>.

Index Terms—Dynamic vehicle routing problem, Neural combinatorial optimization, Distributed vehicle network, Deep reinforcement learning.

I. INTRODUCTION

VEHICLE Routing Problem (VRP) is a classical combinatorial optimization problem, which can be found in many real-world scenarios such as logistics, delivery, and transportation [1], [2]. The traditional static VRP assumes that all customer demands and vehicle information are known before planning [3], which provides some convenience for the solution. However, in practical applications, especially in real-time delivery services where vehicles operate continuously throughout the day [4], the customer requirement often changes dynamically [5], and the vehicle may receive new delivery requests during travel [6], leading to Dynamic Capacitated VRP (DCVRP).

Manually designing an effective heuristic for task allocation is extremely hard since both customer assignment and vehicle

routing are intractable, and they are mutually related to each other. Simple heuristics like first-come first-served or nearest first are computationally efficient, but their performance is usually not good enough to satisfy the demand for real-time and large-scale processing capability of modern logistics systems [7], [8].

In recent years, Neural Combinatorial Optimization (NCO) methods based on encoder-decoder architecture have made significant progress in solving VRP [9]–[11]. Specifically, by combining Reinforcement Learning (RL) and neural networks, NCO methods can generate high-quality solutions quickly, showing great potential to outperform manually designed heuristics [12]–[15]. However, existing NCO methods focus mainly on handling VRP in static scenarios, ignoring dynamics in real applications [16]. Applying these methods to dynamic scenarios is not straightforward, primarily for two reasons. 1) Static NCO methods conduct one-time planning with complete information, failing to consider the time factor. In dynamic scenarios, customer requests emerge continuously, necessitating decisions under incomplete information [17]. 2) Dynamic scenarios require all vehicles to operate continuously throughout the whole working day rather than waiting at the depot [18], [19]. Effective coordination must account for real-time vehicle positions and capacities, creating complex fleet management challenges [20]. In contrast, static NCO methods usually simulate multiple vehicles by sequentially constructing a complete route for each vehicle. Such a solution construction method works for static settings but fails in dynamic VRP, where all vehicles must be coordinated simultaneously in real time. Therefore, static sequential construction of multiple routes cannot be directly extended to dynamic scenarios.

A few NCO methods have attempted to solve dynamic VRP, whenever a new customer arrives during service, the encoder dynamically incorporates the new node by updating the feature set and applying the mask accordingly [21], [22]. However, these methods still exhibit deficiencies when handling vehicle coordination, primarily manifested in two common strategies. 1) Expert knowledge-based vehicle allocation strategy, as shown in Fig. 1(a), relies on predefined rules to select vehicles, such as choosing the shortest-travel-time vehicle or sequential allocation [23], [24]. These methods make quick decisions but overly focus on immediate benefits while sacrificing long-term efficiency. For instance, selecting the nearest vehicle may force deviation from planned routes, compromising future task completion. Such rules require substantial domain knowledge,

Corresponding author: Ya-Hui Jia.

Yang Wang, Ya-Hui Jia, and Zhenhong Lin are with School of Future Technology, South China University of Technology, Guangzhou 510641, China. (email: ftwangyang@mail.scut.edu.cn, jia.yahui@foxmail.com, Zhenhong@scut.edu.cn)

Qiang Yang is with the School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing 210044, China (email: mmmymq@126.com).

Feng-Feng Wei and Wei-Neng Chen are with School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China. (email: fengfengwei@scut.edu.cn, cwnraul634@aliyun.com)

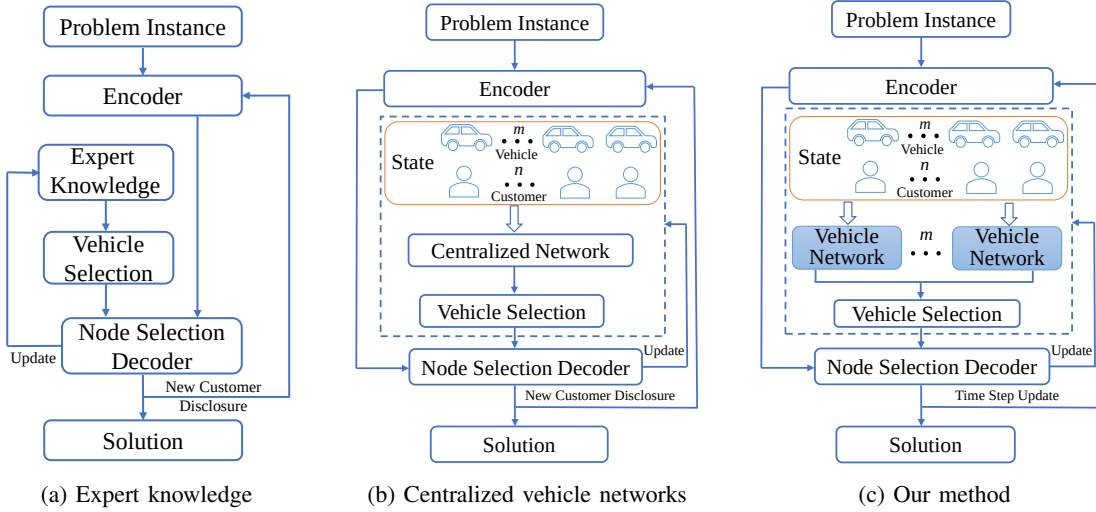


Fig. 1: Three different methods of vehicle allocation in the NCO methods based on encoder-decoder architecture.

yet still struggle to adapt to dynamic environments. 2) Centralized vehicle network architecture, illustrated in Fig. 1(b), uses the same parameters for all vehicles despite their different states, creating an inherent decision conflict [25], [26]. When vehicles operate in diverse conditions with varying routes, these shared parameters cannot simultaneously optimize for all vehicle situations, resulting in decisions that favor certain vehicle scenarios while performing poorly in others, ultimately compromising model robustness.

To address the above challenges, we propose a Distributed Vehicle Network with Decision Aggregation (DVNDA) to solve DCVRP, as shown in Fig. 1(c). Specifically, DVNDA equips each vehicle with an independent neural network to realize parallel decision-making of vehicles and synthesize the decision-making information of all the vehicles at each decision-making step to achieve efficient vehicle cooperative allocation. The contributions of this paper can be summarized as follows:

- 1) We construct a time-driven DCVRP environment that takes into account real-time disclosure of customers. Customer demand arrivals are modeled as stochastic and time-dependent, which allows the environment to capture the dynamic nature of customer orders in reality.
- 2) We design a new algorithm called DVNDA to improve the efficiency of cooperative allocation among vehicles.
 - Unlike existing methods, DVNDA equips each vehicle with an independent neural network, enabling it to make independent decisions based on information about the status of all vehicles and current customers.
 - DVNDA enables collaboration between vehicles by aggregating the decisions of all networks in each step to select the appropriate vehicle to serve the customer.

Extensive experiments on synthetic and real-world instances demonstrate that DVNDA consistently achieves the lowest routing cost across all tested scales and dynamic rates. Furthermore, DVNDA exhibits strong cross-scale and cross-

distribution generalization.

The remainder of this paper is structured as follows. Section II presents the problem formulation and reviews the existing literature on NCO methods for static/dynamic VRPs. Section III provides a comprehensive description of the DVNDA methodology. Section IV provides the experimental analysis. Finally, Section V summarizes the main conclusions and outlines directions for future work.

II. BACKGROUND

A. Dynamic Capacitated Vehicle Routing Problems

DCVRP is a variant of VRP that seeks to find the optimal routing plan for a fleet of vehicles in a dynamic environment [27]. Unlike the static VRP, in DCVRP, new customers keep coming dynamically during the service process, making the problem closer to some real-world applications such as real-time logistics, dynamic traffic routing, or emergency service management. The following formulation extends the CVRP formulation used in [10] by incorporating temporal constraints (customer disclosure times and a planning horizon), following the dynamic modeling conventions established in [23], [28]. DCVRP can be defined on a complete graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. $\mathcal{V} = \{0, 1, 2, \dots, n\}$ is the set of nodes, where node 0 represents the depot and nodes 1 to n represent customers. $\mathcal{E}$ is the set of edges. Each edge (i, j) is associated with a non-negative value d_{ij} , representing the travel cost between nodes i and j . Each vehicle k has a capacity limit Q_k , representing its maximum load capacity. The goal is to meet customer demands while minimizing the total travel cost. The mathematical formulation is:

$$\min \sum_{k=1}^m \sum_{i=0}^n \sum_{j=0}^n d_{ij} x_{ijk} \quad (1)$$

subject to:

$$\sum_{k=1}^m \sum_{j=0}^n x_{ijk} = 1, \quad \forall i \in \mathcal{V} \setminus \{0\} \quad (2)$$

$$\sum_{j=0}^n x_{0jk} = 1, \quad \forall k \in \{1, \dots, m\} \quad (3)$$

$$\sum_{i=0}^n x_{i0k} = 1, \quad \forall k \in \{1, \dots, m\} \quad (4)$$

$$\sum_{i=1}^n q_i \sum_{j=0}^n x_{ijk} \leq Q_k, \quad \forall k \in \{1, \dots, m\} \quad (5)$$

$$\sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}} x_{ijk} \leq |\mathcal{S}| - 1, \quad \forall \mathcal{S} \subset \mathcal{V}, |\mathcal{S}| \geq 2 \quad (6)$$

$$t_j \geq t_i + z_i + d_{ij} - \mu(1 - x_{ijk}), \quad \forall i, j \in \mathcal{V} \quad (7)$$

$$t_j - \sum_{i \in \mathcal{V}} x_{ij} d_{ij} \leq T, \quad \forall j \in \mathcal{V} \quad (8)$$

$$t_i \geq a_i, \quad \forall i \in \mathcal{V} \setminus \{0\} \quad (9)$$

The decision variables are $x_{ijk} \in 0, 1$ indicating if vehicle k traverses edge (i, j) , and $t_i \geq 0$ denotes the time vehicle starts serving customer i . m is the total number of vehicles available. q_i is demand of customer node i . a_i is the disclosure time of customer node i , indicating the earliest time at which node i becomes available for service. z_i is the service time required at customer node i . $\mathcal{S}$ denotes a subset of customer nodes. μ is a sufficiently large constant used to linearize constraints. T is the total planning horizon and a_i is the reveal time of node i . Constraint (2) ensures each customer is visited exactly once. Constraints (3)-(4) enforce depot departure and return. Constraint (5) maintains vehicle capacity limits. Constraint (6) eliminates subtours. Constraint (7) ensures that if vehicle k travels from node i to node j , then the arrival time at node j must be at least the arrival time at node i plus the service time at node i plus the travel time from i to j . Constraint (8) ensures that each customer must be scheduled for service before the planning horizon T . The constraint (9) enforces dynamic customer availability by requiring that a vehicle cannot begin service until the customer becomes available.

The dynamic nature of DCVRP is quantified using two primary metrics [28]: dynamic node ratio φ and average node revelation time γ that are defined as:

$$\varphi = \frac{n'}{n}, \quad \gamma = \frac{1}{n} \sum_{i=1}^n \frac{a_i}{T}, \quad (10)$$

where n' is the number of dynamically revealed nodes and n is the total number of nodes. Intuitively, these two metrics capture distinct aspects of the dynamic challenge faced in real-world logistics. The ratio φ reflects the proportion of “urgent” or “ad-hoc” orders, those not known at the beginning of the planning horizon. The metric γ characterizes the temporal distribution of these arrivals. A low γ indicates that dynamic orders tend to appear early in the planning horizon, giving vehicles more time to replan and adapt their routes, whereas a high γ means late-arriving requests that leave limited room for replanning.

B. Related Work

In this section, we review the existing literature on solving static/dynamic VRPs using the NCO method.

Static VRP. In the study of VRP and its variants, a variety of NCO methods have emerged in recent years [29]. The most notable among these are the attention model introduced by Kool *et al.* [10] and the policy optimization with multiple optima by Kwon *et al.* [11], which have since become the cornerstone of most subsequent NCO methods. Building on these foundational frameworks, Li *et al.* [30] proposed a heterogeneous attention and graph convolutional network method to solve the asymmetric pickup and delivery problem and showed that it outperformed traditional and learning-based baselines with strong generalization. Wang *et al.* [31] proposed a deep reinforcement learning-based neural heuristic with a two-stage attention encoder to solve vehicle routing problems with backhauls and demonstrated superior solution quality and generalization over existing methods. Lin *et al.* [32] proposed a multi-agent RL framework to realize the simultaneous management of multiple vehicle operations. Zhang *et al.* [24] proposed a Multi-Agent Attention Model (MAAM) for solving the routing problems with soft time windows. MAAM sequentially selects vehicles to serve customers at each step. Li *et al.* [25] proposed a deep RL method (LiDRL) to solve the heterogeneous capacitated VRP. The method contains a vehicle selection decoder and a node selection decoder, which can automatically select suitable vehicles and nodes for path construction at each step. Existing research has shown that the multi-agent RL methods have significant advantages in effectively solving routing problems by enabling collaborative decision-making [33]–[35]. However, a common limitation of these methods is that they primarily focus on static scenarios, lacking practicality for dynamic environments.

Dynamic VRP. Zhang *et al.* [17] proposed a deep reinforcement learning method with two encoder-decoder models to solve the dynamic traveling salesman problem. It effectively captures environmental changes and achieves superior real-time performance compared with existing methods. However, it can only solve the problems with one vehicle. Bono *et al.* [23] designed Multi-Agent Routing using Deep Attention Mechanisms (MARDAM) to solve the dynamic routing problems. MARDAM achieves vehicle assignment by selecting the vehicle with the smallest cumulative travel time at each time step. Xiang *et al.* [26] proposed an Attention Model with Centralized Vehicle Networks (AMCVN) to solve the dynamic pickup and delivery problem with crowd shippers. AMCVN integrates a centralized vehicle network that observes the state information of all the vehicles and selects the vehicles to serve the customers at each time step. Zhang *et al.* [36] proposed an end-to-end learning framework based on the attention mechanism to address the dynamic pickup and delivery problem. Nevertheless, their method does not account for cooperative optimization among vehicles. Sultana *et al.* [37] proposed an RL-based method to solve dynamic routing with time windows, which uses a centralized allocation heuristic to coordinate vehicle allocation.

In summary, while research has advanced in both static

and dynamic VRP using NCO methods, significant limitations remain in the vehicle coordination mechanisms for DCVRP. First, expert knowledge-based allocation rules operate in isolation, preventing the sharing of inter-vehicle information and degrading overall model performance. Second, even though centralized vehicle networks can access global state information, their shared parameter architecture inherently fails to accommodate diverse vehicle conditions, creating decision biases that ultimately undermine solution quality and model robustness. The proposed DVNDA addresses these limitations by giving each vehicle its own independently parameterized network and coordinating them through a lightweight, parameter-free aggregation mechanism. Unlike centralized networks that force all vehicles to share one model, DVNDA allows each vehicle to specialize in its own context. Unlike expert-rule methods, DVNDA learns the allocation strategy from data rather than relying on hand-crafted heuristics.

III. METHODOLOGY

A. Multi-Agent Markov Decision Process for DCVRP

DCVRP is modeled as a time-driven multi-agent decision process, where “multi-agent” refers to a collaborative system in which each vehicle acts as an autonomous agent making independent decisions. All agents interact within a shared dynamic environment, where real-time customer requests are progressively disclosed, and the status of all vehicles is visible to every agent. The planning horizon T is divided into β equal intervals with boundaries $0 = T_1 < \dots < T_\beta$. At the beginning of each interval $T_r \in \{T_1, \dots, T_{\beta-1}\}$, vehicles independently generate routing decisions based on the current global state, and a decision aggregation mechanism synthesizes these plans to assign tasks and coordinate execution.

State. The system state $S^{(r)}$ at the beginning of interval r encompasses synchronized vehicle states and customer information available for the current interval:

$$S^{(r)} = \left(\{U_k^{(r)}\}_{k=1}^m, C_{\text{available}}^{(r)}, C_{\text{done}}^{(r)} \right), \quad (11)$$

where $U_k^{(r)}$ represents vehicle k 's state at time T_r . $C_{\text{available}}^{(r)}$ denotes the set of customers available for service assignment at T_r , including both newly revealed customers and those not completed in previous intervals. These customers still had no vehicle dispatched to serve them by the end of the previous interval. $C_{\text{done}}^{(r)}$ denotes the set of customers that have been served by the end of the previous interval, including customers that have been served, customers that are currently being served, and customers that have dispatched vehicles and are en route.

Each vehicle k maintains a synchronized state vector:

$$U_k^{(r)} = \left(T_r, \text{pos}_k^{(r)}, \text{cap}_k^{(r)} \right), \quad (12)$$

where $\text{pos}_k^{(r)}$ is vehicle k 's position and $\text{cap}_k^{(r)}$ is vehicle k 's remaining capacity.

Action. At T_r , the action space represents route assignments for all available customers:

$$A^{(r)} = \{\text{route}_k^{(r)}\}_{k=1}^m, \quad (13)$$

where $\text{route}_k^{(r)}$ is the route assigned to vehicle k for the interval r , constructed over $C_{\text{available}}^{(r)}$. Each vehicle departs from its current position to execute the assigned route and serve the corresponding customers.

Transition. The transition from interval r to interval $r + 1$ occurs at the end of interval r , when all vehicles scheduled are either completed, in service, or already dispatched and en route. Customers who were already served, are currently being served, or have vehicles dispatched to them during interval r remain fixed in the schedule for interval $r + 1$. All other customers are replanned, and any newly revealed customers are added to the task set. Algorithm 1 outlines this procedure.

Algorithm 1: Interval Transition from r to $r + 1$

Input: Current state $S^{(r)}$, planned path $A^{(r)}$

Output: Next state $S^{(r+1)}$

- 1 Execute routes in $A^{(r)}$
 - 2 **for** each customer $i \in C_{\text{available}}^{(r)}$ **do**
 - 3 Calculate the departure time $\mathcal{D}_i^{\text{start}}$ and completion time $\mathcal{D}_i^{\text{done}}$ based on planned path $A^{(r)}$
 - 4 **end**
 - 5 $C_{\text{done}}^{(r+1)} \leftarrow \{i \mid (\mathcal{D}_i^{\text{done}} \leq T_{r+1}) \vee (\mathcal{D}_i^{\text{start}} < T_{r+1} \wedge \mathcal{D}_i^{\text{done}} > T_{r+1})\}$
 - 6 $C_{\text{available}}^{(r+1)} \leftarrow C_{\text{available}}^{(r)} \cup \{i \mid a_i > T_r \wedge a_i \leq T_{r+1}\} - C_{\text{done}}^{(r+1)}$
 - 7 **for** each vehicle $k = 1, 2, \dots, m$ **do**
 - 8 Calculate position $\text{pos}_k^{(r+1)}$ and remaining capacity $\text{cap}_k^{(r+1)}$
 - 9 $U_k^{(r+1)} \leftarrow (T_{r+1}, \text{pos}_k^{(r+1)}, \text{cap}_k^{(r+1)})$
 - 10 **end**
 - 11 $S^{(r+1)} \leftarrow (\{U_k^{(r+1)}\}_{k=1}^m, C_{\text{available}}^{(r+1)}, C_{\text{done}}^{(r+1)})$
 - 12 **return** $S^{(r+1)}$
-

Reward. The reward function evaluates the overall solution quality based on the final complete routing solution:

$$R(S^{(T)}, A^{(T)}) = - \left(\sum_{k=1}^m \sum_{i=0}^n \sum_{j=0}^n d_{ij} x_{ijk} + \alpha |C_{\text{unserved}}| \right), \quad (14)$$

where α is the penalty coefficient for unserved customers, and C_{unserved} represents the set of unserved customers. Due to the dynamic nature of the environment and the potential suboptimality of the policy, some tasks may remain uncompleted within the regular time horizon. In such cases, the total traveled distance may be much shorter than if all customers were served. We do not encourage this behavior because, in extreme cases, a policy could choose to perform no tasks at all and achieve a total distance of zero. To prevent this, the penalty ensures that any solution leaving customers unserved will receive a reward lower than that of serving all customers, guiding the policy toward completing as many deliveries as possible.

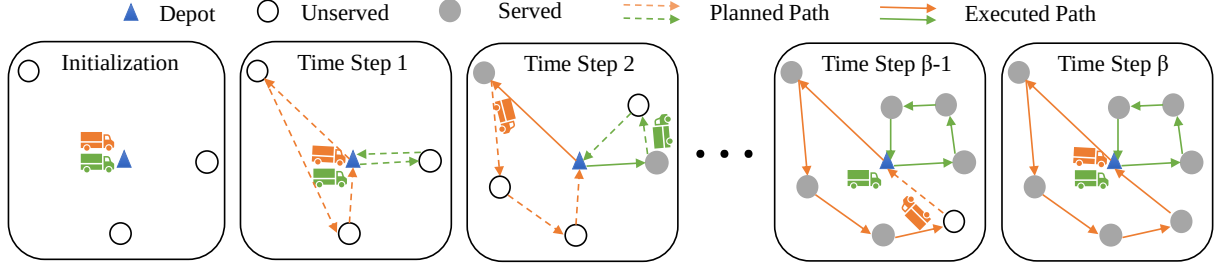


Fig. 2: DVNDA workflow for 2 vehicles and 7 customers.

Policy. The policy π determines the optimal vehicle-customer assignment given the current state. The objective is to minimize the total travel cost and unserved customer penalties:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[R(S^{(T)}, A^{(T)}) \mid \pi \right]. \quad (15)$$

B. Time-Driven DCVRP Environment

Unlike previous methods [23], [26] that assign each vehicle an independent timeline over the planning horizon, thereby limiting coordination and restricting applicability in real-world scenarios. Our framework utilizes a unified, synchronized time interval to inform global decision-making. By aligning all vehicles to the same interval boundaries $\{T_1, \dots, T_\beta\}$, the system avoids inconsistencies caused by fragmented timelines. This design preserves vehicle-level flexibility in task execution while ensuring fleet coordination and resource optimization, resulting in routing strategies that are both operationally efficient and practically deployable.

Specifically, the DCVRP environment operates through synchronized interval-based planning, where each interval functions as a distinct planning phase. At the beginning of each interval, the system collects all newly disclosed customer information that has accumulated since the previous interval, incorporating these customers into the updated routing plan. Vehicles update their current positions and capacity information to formulate a new routing plan. Fig. 2 provides an overview of the DVNDA decision-making process for a scenario with 2 vehicles and 7 customers under a time-driven framework.

C. DVNDA Architecture

The DVNDA model comprises three main components: an encoder for feature extraction, a distributed vehicle network with decision aggregation for vehicle selection, and a node selection decoder for customer assignment, as illustrated in Fig. 3. The model operates at the start of each planning interval $[T_r, T_{r+1}]$, constructing complete routes for all vehicles based on currently available customer information. For clarity in presenting the solution procedure, we use the interval $[T_r, T_{r+1}]$ as a representative example to illustrate the detailed operation of each model component (encoder, vehicle network, and decoder) in the route construction process.

1) *Encoder:* For all nodes, including the depot and currently visible customers, the attribute features $\mathbf{v}_i$ are mapped into a higher-dimensional space as the initial embeddings:

$$\mathbf{h}_i^{(0)} = \mathbf{W}_i \mathbf{v}_i, \quad (16)$$

where $\mathbf{W}_i$ is a trainable embedding matrix, $\mathbf{v}_i$ contains node attributes (coordinates, demand, reveal time, and service time).

To ensure that each vehicle only considers customers within its current visibility, we employ vehicle-specific temporal masks. For vehicle k , a mask $\mathbf{M}_{i,k}$ is applied such that:

$$\mathbf{h}_{i,k}^{(0)} = \mathbf{h}_i^{(0)} \odot \mathbf{M}_{i,k}, \quad (17)$$

where $\odot$ is the Hadamard product. $\mathbf{M}_{i,k} = 1$ if customer i is revealed and available to vehicle k , and 0 otherwise. This masking operation ensures that each vehicle's encoder only considers customers that have been disclosed by the current interval r , thus preventing any influence from future (unrevealed) customers.

The initial embedding features are processed through the L encoder layer consisting of a Multi-Head Attention (MHA) sublayer followed by a Feed Forward (FF) sublayer, with residual connections and batch normalization applied at each step:

$$\hat{\mathbf{h}}_{i,k}^{(l)} = \text{BN} \left(\mathbf{h}_{i,k}^{(l)} + \text{MHA}(\mathbf{h}_{i,k}^{(l)}) \right), \quad (18)$$

$$\mathbf{h}_{i,k}^{(l+1)} = \text{BN} \left(\hat{\mathbf{h}}_{i,k}^{(l)} + \text{FF}(\hat{\mathbf{h}}_{i,k}^{(l)}) \right), \quad (19)$$

where l denotes the layer index, and BN represents batch normalization.

After L layers, the final node embeddings $\mathbf{h}_{i,k}^{(L)}$ represent the enriched features of customers. The graph embedding $\bar{\mathbf{h}}_{G,k}$ is obtained by averaging the node embeddings:

$$\bar{\mathbf{h}}_{G,k} = \frac{1}{C_{\text{available}}^{(r)}} \sum_{i \in C_{\text{available}}^{(r)}} \mathbf{h}_{i,k}^{(L)}. \quad (20)$$

2) *Distributed Vehicle Network with Decision Aggregation:* The DVNDA architecture adopts a distributed design in which each vehicle k is equipped with an independently parameterized neural network to evaluate its action based on the current state and context. These parallel evaluations are then aggregated to determine which vehicle to dispatch next during route construction. Here, “distributed” specifically refers to the use of independent network parameters for each vehicle, rather than a shared centralized model.

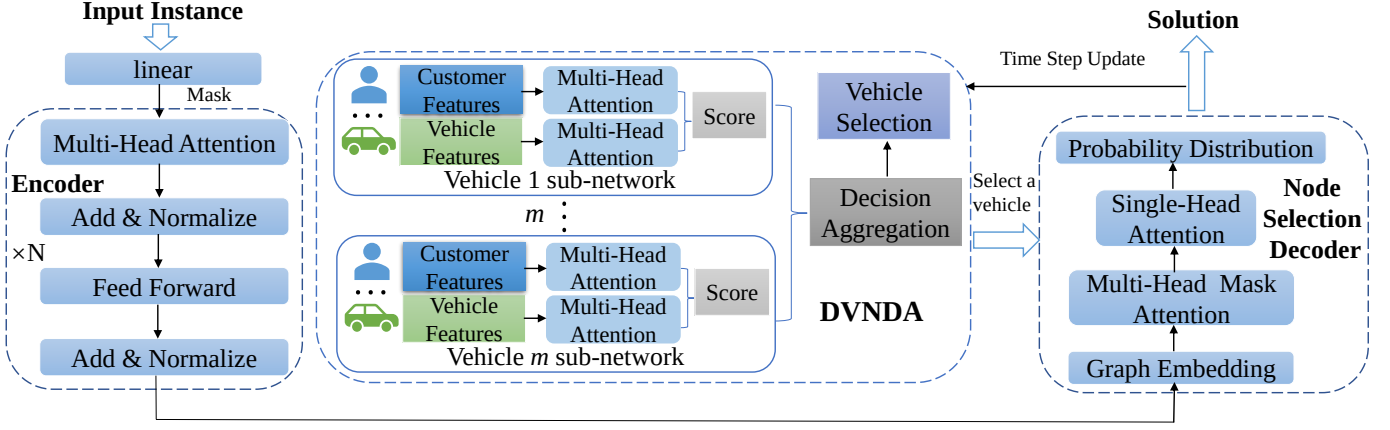


Fig. 3: The structure of DVNDA.

Each vehicle k maintains its state representation s_k , including current position, timeline, and remaining capacity. To capture both vehicle and customer information in a temporally aware manner, the vehicle and customer features are embedded as:

$$e_k^o = W^o s_k, \quad e_{i,k}^c = W^c c_{i,k}, \quad (21)$$

where e_k^o is the embedded vector of vehicle k . W^o and W^c are trainable linear projection matrices. $e_{i,k}^c$ is the embedded vector of the customer i for vehicle k . $c_{i,k}$ represents customer i 's features as perceived by vehicle k .

Each vehicle's network then processes both fleet and customer contexts through MHA, taking into account only the customers visible to that vehicle at its current state:

$$f_k = \text{MHA}(W_Q^{\text{veh}} e_k^o, W_K^{\text{veh}} E^o, W_V^{\text{veh}} E^o, M^o), \quad (22)$$

$$w_k = \text{MHA}(W_Q^{\text{cus}} e_{i,k}^c, W_K^{\text{cus}} E_k^c, W_V^{\text{cus}} E_k^c, M_k^c), \quad (23)$$

where f_k is the vehicle context vector for vehicle k . w_k is the customer context vector for vehicle k . E^o is the embedding of all vehicles. E_k^c is the embeddings of customers visible to vehicle k . M^o is the vehicle visibility mask. M_k^c is the customer visibility mask. W_Q^{veh} , W_K^{veh} , and W_V^{veh} are projection matrices for vehicle queries, keys, and values. W_Q^{cus} , W_K^{cus} , and W_V^{cus} are projection matrices for customer queries, keys, and values.

The context vectors are then combined to compute the scores for vehicle assignment:

$$I_k = [e_k^o, f_k, w_k], \quad (24)$$

$$h_k = \text{ReLU}(W_1 I_k + b_1), \quad (25)$$

$$l_k = W_2 h_k + b_2, \quad (26)$$

where l_k is the score that represents the priority of vehicle k for the next customer assignment. W_1 and W_2 are learnable weight matrices. b_1 and b_2 are bias terms. A masking step is incorporated to ensure only feasible and available vehicles are considered. If vehicle k has no remaining capacity or is otherwise unable to take a new assignment, its score l_k is set to $-\infty$.

Unlike node selection, which operates over a large customer space and benefits from stochastic sampling for exploration, vehicle selection involves only m candidates whose states already vary across construction steps, providing natural diversity. Therefore, the decision aggregation selects the vehicle k^* with the highest score:

$$k^* = \arg \max_k l_k. \quad (27)$$

The vehicle k^* will be assigned the next customer in the route construction process. This selection is an intermediate decision within the DVNDA planning. It identifies which vehicle should receive a customer next in order to build out the routes one customer at a time.

3) *Node Selection Decoder*: Once a vehicle k^* is chosen, the node selection decoder chooses which customer that vehicle will serve next. The decoder operates from the perspective of the selected vehicle k^* , considering only the customers visible to k^* in the current interval. A context vector is formed by combining the global context (graph embedding) and the embedding of the last visited node:

$$H_c^T = [\bar{h}_{G,k^*}, h_{\text{last},k^*}^{T-1}], \quad (28)$$

where $\bar{h}_{G,k^*}$ is the graph embedding from vehicle k^* 's perspective, T denotes the current construction step in interval r and $h_{\text{last},k^*}^{T-1}$ is the embedding of the last visited node by k^* . The decoder then computes attention between this context vector and the embeddings of all candidate customers for vehicle k^* :

$$\hat{H}_c^T = \text{MHA}(W_Q^{\text{dec}} H_c^T, W_K^{\text{dec}} h_{i,k^*}^{(L)}, W_V^{\text{dec}} h_{i,k^*}^{(L)}), \quad (29)$$

where W_Q^{dec} , W_K^{dec} , W_V^{dec} are learned projection matrices for the query, key, and value. This attention yields a compatibility score for each candidate customer i (unserved and visible to k^*). We compute these scores and apply a tanh scaling to control their range:

$$u_T = C \cdot \tanh \left(\frac{(W_Q^{\text{comp}} \hat{H}_c^T)(W_K^{\text{comp}} h_{i,k^*}^{(L)})^\top}{\sqrt{\text{dim}_k}} \right), \quad (30)$$

where $C = 10$ is the scaling coefficient that controls the compatibility scores [10]. dim_k is the dimensionality of the keys

and queries in the compatibility attention. $\mathbf{W}_Q^{\text{comp}}$ and $\mathbf{W}_K^{\text{comp}}$ are learned embedding matrices. The probability distribution $\bar{p}_T$ over unserved nodes is obtained via softmax, masking previously visited nodes to ensure feasibility:

$$\bar{p}_T = \text{softmax}(u_T). \quad (31)$$

The DVNDA then iteratively repeats the vehicle selection and node selection steps to construct routes until all customers available in the interval are assigned or all vehicles can no longer serve additional requests.

4) *Interval update*: Within the current interval r , once route planning is completed, the system transitions to the next interval. At the beginning of this new interval, the set of available customers consists of newly revealed customers together with unserved customers from the previous interval. The model then updates the embeddings of the newly added nodes:

$$\mathbf{h}_{i,k,r}^{(L)} = \text{Encoder}(\mathbf{h}_{i,k,r}^{(0)}, \mathbf{M}_{i,k,r}^{\text{updated}}), \quad (32)$$

where $\mathbf{h}_{i,k,r}^{(L)}$ is the updated embedding after L layers of the encoder, used for route planning in the next interval. Encoder denotes the sequence of operations described in Eqs. (16)-(20). $\mathbf{h}_{i,k,r}^{(0)}$ is the initial embedding of customer i for vehicle k in interval r . $\mathbf{M}_{i,k,r}^{\text{updated}}$ is the updated mask matrix for vehicle k in interval r , including all known customers, i.e., both previously known and newly revealed ones.

In this way, the DVNDA architecture operates in a synchronous interval manner. Within each interval, DVNDA constructs routes until route planning is completed, then updates at the next interval by incorporating both newly revealed and previously unserved customers into the subsequent planning process.

D. Training Method

To train the proposed DVNDA for DCVRP, we employ the REINFORCE algorithm augmented with a rollout baseline, as detailed in prior studies [10], [38], [39]. The overall training procedure is presented in Algorithm 2. The training framework contains two separate sets of parameters: θ^p for the policy network and θ^{BL} for the baseline network. After each training epoch, the policy network parameters θ^p are updated through gradient descent using the loss function defined by:

$$\nabla \mathcal{L}(\theta^p) = \mathbb{E}_{\pi \sim p_{\theta^p}(\pi|S)} [(R_\pi - R_{\pi^{\text{BL}}}) \nabla_{\theta^p} \log p_{\theta^p}(\pi|S)], \quad (33)$$

where π represents the routing policy generated by the policy network with parameters θ^p , and π^{BL} denotes the policy produced by the baseline network with parameters θ^{BL} . $p_{\theta^p}(\pi|S)$ denotes the probability of the sequence of decisions made by the policy.

IV. EXPERIMENTS

All methods are trained and run on a single NVIDIA 4090D GPU with 24GB memory and Intel(R) Core(TM) i9-12900K CPU at 3.2GHz.

Algorithm 2: DVNDA Training

Input: Epochs $\mathcal{I}$, Batch size $\mathcal{B}$, Steps $\mathcal{N}$, Intervals β , Significance ϕ
Output: Parameters θ^p

```

1 for epoch = 1, ...,  $\mathcal{I}$  do
2   for step = 1, ...,  $\mathcal{N}$  do
3     for batch = 1, ...,  $\mathcal{B}$  do
4       Generate instance with customer
         disclosures  $\{a_i\}$ 
5       Initialize interval boundaries  $\{T_1, \dots, T_\beta\}$ 
6       Initialize vehicle states at depot
7       for interval  $r = 1, \dots, \beta$  do
8         Encode customer features using Eq.
           (32)
9         for vehicle  $k = 1, 2, \dots, m$  do
10          Compute scores for vehicle
            selection using Eq. (27)
11          Select customers sequentially using
            Eq. (31)
12          Plan the routes for the current
            interval
13        end
14      end
15      Compute reward using Eq. (14)
16       $\pi_i \leftarrow \text{SampleRollout}(\text{instance}, p_\theta)$ 
17       $\pi_i^{\text{BL}} \leftarrow \text{GreedyRollout}(\text{instance}, p_{\theta^{\text{BL}}})$ 
18      Calculate gradient  $\nabla \mathcal{L}(\theta^p)$  using Eq. (33)
19       $\theta^p \leftarrow \text{Adam}(\theta^p, \nabla \mathcal{L}(\theta^p))$ 
20    end
21  end
22  if  $t\text{-test}(p_{\theta^p}, p_{\theta^{\text{BL}}}) < \phi$  then
23     $\theta^{\text{BL}} \leftarrow \theta^p$ 
24  end
25 end
26 return  $\theta^p$ 

```

A. Experimental Setting

Node Revelation Process. Node revelations are assumed to follow a Poisson distribution, characterized by the rate parameter λ [40]. The probability of a node being revealed at time t is given by:

$$P(t) = \frac{\lambda^t e^{-\lambda}}{t!}, \quad (34)$$

where λ represents the average rate of node revelations over the planning horizon.

Training Setup. We train the model on DCVRP instances with dynamic degrees of 0.1, 0.25, 0.5, and 0.75. For the smaller case with $n = 20$ customers and $m = 4$ vehicles, training is performed with a batch size of 100 for 100 epochs and 1000 iterations per epoch. For the larger cases with $n = 35$ and 50 customers ($m = 7$ and 10 vehicles, respectively), training uses the batch size of 50 for 100 epochs, with 500 iterations per epoch. The learning rate of the model is set to 0.0001. The penalty coefficient α is set to 5. The time horizon is set to $T = 480$, corresponding to an 8-hour working day in minutes. The rate parameter λ is set to $\frac{1+T}{2}$. The time interval β is set

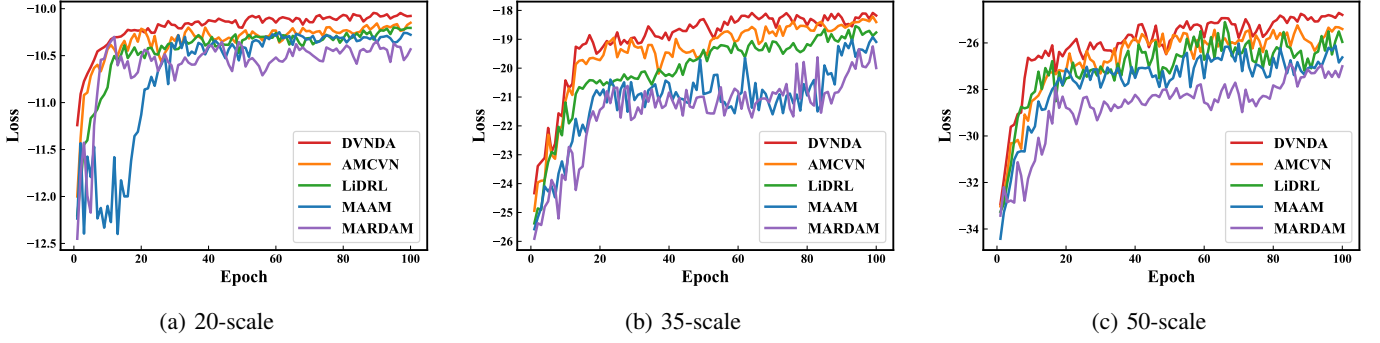


Fig. 4: Convergence behavior of different algorithms.

to 10. The vehicle capacity is 150 with customer demands in [5, 41], ensuring that each vehicle can serve multiple customers per trip but cannot cover all customers alone, thus necessitating fleet coordination. The traveling speed is 1, the service time is uniformly sampled from [10, 31], the ratio $m = n/5$ follows [23], and the coordinate range is uniformly distributed within the $[0, 1]$ unit square.

Test Setup. In instances of different scales, the number of vehicles m is set as $m = \frac{n}{5}$, in accordance with the setting adopted in [23].

Model Setup. The Transformer architecture is employed [41], consisting of 3 encoder layers, each with 8 attention heads. The model's initial embedding dimension is 128, and the FF layer dimension is 512. Training is performed using a rollout baseline with 3 rollouts and a threshold set to 0.05.

Baselines. To evaluate the effectiveness of our proposed method, we compare greedy and NCO methods with state-of-the-art vehicle selection strategies (MARDAM, MAAM, LiDRL, AMCVN). Since no existing NCO method directly addresses DCVRP under our problem formulation, we integrate only the vehicle selection strategy of each baseline into our unified time-driven framework, while keeping all other components (encoder, node decoder, training procedure, and dynamic environment) identical. This controlled setup ensures a fair comparison by isolating the effect of the vehicle coordination mechanism.

- **Greedy:** Every time a vehicle becomes idle, it is assigned to execute the nearest available task.
- **MARDAM [23]:** Every time a vehicle becomes idle, an available task is selected and assigned to it through the neural network model.
- **MAAM [24]:** Vehicles are selected in a round-robin manner to serve customers at each step.
- **LiDRL [25]:** The selection probability of each vehicle is calculated at each step by the collaboration of a vehicle feature embedding network and a route feature embedding network.
- **AMCVN [26]:** All vehicle status information is monitored through a centralized vehicle network, and the selection probability of each vehicle is calculated at each step through this network.

In terms of training time, each iteration of the DVNDA method took about 30 minutes for the scale of 20, and around

25 minutes for the scales of 35 and 50. Fig. 4 illustrates the convergence behavior of five different algorithms over 100 training epochs. The results demonstrate that the DVNDA achieves superior performance with the lowest final loss value compared to AMCVN, LiDRL, MAAM, and MARDAM. DVNDA exhibits more stable convergence with fewer fluctuations throughout the training process, especially in later epochs.

Performance Metrics. The performance of each method is measured across three key aspects:

- **Cost:** Total path length of all vehicles, reported as mean and standard deviation. In the training phase, this includes unserved customer penalties, while the inference phase cost excludes these penalties.
- **Quality of Service (QoS):** Since vehicles operate on single round trips, this is evaluated as the proportion of customers served. Higher QoS indicates better service in dynamic environments.
- **Inference time:** Total computational time required to solve all test instances. Since NCO methods can solve all test instances in a single parallel run on the GPU, while the Greedy method runs sequentially on the CPU, it is not fair for Greedy. Thus, we do not report the average solving time of Greedy.

B. Performance on Synthetic Instances

Overall performance. Table I shows the path length, QoS, and inference time of each method at different dynamic rates (0.1, 0.25, 0.5, 0.75) for synthetic instances ($n = 20, 35, 50$). As the dynamic rate increases, the cost of all methods typically increases. DVNDA demonstrates strong robustness at all dynamic rates and can maintain a high level of QoS while keeping costs low in dynamic environments.

Greedy VS DVNDA. DVNDA clearly outperforms the greedy heuristic in both cost and service quality across nearly all scenarios. For instance, with $\varphi = 10\%$ and $n = 50$ customers, DVNDA achieves approximately a 11.8% reduction in total route length compared to greedy (18.89 vs 21.41), with both methods serving essentially all customers. Even at higher dynamic rates ($\varphi = 50\%$), DVNDA maintains a significant cost advantage over greedy (25.31 vs 26.71) while preserving 100% QoS. At the highest dynamic rate ($\varphi = 75\%$), DVNDA still

TABLE I: Detailed comparisons between Greedy, MARDAM, MAAM, LiDRL, AMCVN, and DVNDA on synthetic instances (100 instances). **Bold** means the best results.

φ	Method	$n = 20$			$n = 35$			$n = 50$		
		Cost	QoS	Time	Cost	QoS	Time	Cost	QoS	Time
10%	Greedy	9.07 $\pm$ 1.12	99.90%	—	15.95 $\pm$ 1.44	99.95%	—	21.41 $\pm$ 2.11	99.95%	—
	MARDAM	8.91 $\pm$ 1.29	99.95%	1s	15.86 $\pm$ 2.32	99.95%	1s	21.24 $\pm$ 2.96	99.95%	3s
	MAAM	8.83 $\pm$ 1.22	99.80%	1s	15.54 $\pm$ 2.37	99.95%	1s	19.81 $\pm$ 2.24	99.90%	3s
	LiDRL	8.67 $\pm$ 1.27	100%	1s	15.13 $\pm$ 2.42	100%	2s	19.38 $\pm$ 2.31	100%	4s
	AMCVN	8.39 $\pm$ 1.29	100%	1s	15.03 $\pm$ 2.42	100%	2s	18.94 $\pm$ 2.75	100%	4s
	DVNDA	8.31 $\pm$ 1.22	100%	1s	14.94 $\pm$ 2.00	100%	3s	18.89 $\pm$ 2.27	100%	5s
25%	Greedy	9.69 $\pm$ 1.25	99.90%	—	16.96 $\pm$ 1.95	99.95%	—	22.84 $\pm$ 2.10	99.85%	—
	MARDAM	9.85 $\pm$ 1.27	99.80%	1s	16.90 $\pm$ 2.42	99.95%	1s	22.75 $\pm$ 2.81	99.95%	3s
	MAAM	9.68 $\pm$ 1.55	99.65%	1s	16.83 $\pm$ 2.46	99.80%	1s	22.33 $\pm$ 2.69	99.95%	3s
	LiDRL	9.45 $\pm$ 1.24	100%	1s	16.71 $\pm$ 2.53	100%	2s	21.78 $\pm$ 2.75	100%	4s
	AMCVN	9.23 $\pm$ 1.23	100%	1s	16.42 $\pm$ 2.90	100%	2s	21.56 $\pm$ 2.51	100%	4s
	DVNDA	8.95 $\pm$ 1.30	100%	1s	16.14 $\pm$ 2.14	100%	3s	21.21 $\pm$ 2.66	100%	5s
50%	Greedy	11.25 $\pm$ 1.43	99.90%	—	19.63 $\pm$ 2.01	99.95%	—	26.71 $\pm$ 2.48	99.95%	—
	MARDAM	11.45 $\pm$ 1.37	99.85%	1s	19.79 $\pm$ 2.55	99.90%	1s	26.91 $\pm$ 2.87	99.95%	3s
	MAAM	11.32 $\pm$ 1.30	99.40%	1s	19.68 $\pm$ 2.68	99.65%	1s	26.76 $\pm$ 2.78	99.80%	3s
	LiDRL	11.01 $\pm$ 1.45	100%	1s	19.16 $\pm$ 2.49	100%	2s	25.65 $\pm$ 2.98	100%	4s
	AMCVN	10.75 $\pm$ 1.54	100%	1s	19.04 $\pm$ 2.46	100%	2s	25.49 $\pm$ 2.89	100%	4s
	DVNDA	10.47 $\pm$ 1.53	100%	1s	18.90 $\pm$ 2.24	100%	3s	25.31 $\pm$ 2.81	100%	5s
75%	Greedy	12.43 $\pm$ 1.51	99.45%	—	21.55 $\pm$ 2.21	99.95%	—	30.19 $\pm$ 2.77	99.85%	—
	MARDAM	12.81 $\pm$ 1.49	99.85%	1s	21.84 $\pm$ 2.71	99.85%	1s	30.57 $\pm$ 2.48	99.90%	3s
	MAAM	12.72 $\pm$ 1.53	99.95%	1s	21.70 $\pm$ 2.11	99.65%	1s	30.30 $\pm$ 3.11	99.20%	3s
	LiDRL	12.52 $\pm$ 1.62	100%	1s	21.47 $\pm$ 2.85	100%	2s	29.87 $\pm$ 3.07	100%	4s
	AMCVN	12.03 $\pm$ 1.47	100%	1s	21.19 $\pm$ 2.62	100%	2s	29.33 $\pm$ 3.05	100%	4s
	DVNDA	11.78 $\pm$ 1.44	100%	1s	20.98 $\pm$ 2.26	100%	3s	29.00 $\pm$ 2.69	100%	5s

yields a lower total travel cost (29.00 vs 30.19), demonstrating superior efficiency under extreme dynamism.

Greedy relies on instantaneous consideration of local information and cannot plan globally for a long time, resulting in poor overall performance. Especially in high dynamic rates, the greedy method is unable to respond to frequent new customer requests effectively. On the other hand, DVNDA can adjust vehicle allocation more flexibly through the distributed vehicle collaboration mechanism, ensuring efficient service quality in dynamic environments.

NCO methods VS DVNDA. On the synthetic benchmarks, DVNDA consistently outperforms the state-of-the-art NCO methods (MARDAM, MAAM, LiDRL, and AMCVN) in terms of both solution cost and service level, and it does so without incurring additional inference time. For instance, at $\varphi = 75\%$ with $n = 50$, DVNDA attains the lowest total cost of about 29.00 while maintaining 100% QoS, whereas the next best learned method (AMCVN) has a higher cost of about 29.33 and others like MARDAM and MAAM incur substantially larger costs of about 30.57 and 30.30, respectively, and even slight QoS drops. Notably, DVNDA's inference speed is comparable to baseline methods. For example, on $n = 50$ instances with $\varphi = 75\%$, DVNDA completes all test cases in approximately 5 seconds, which is similar to other learned methods. Furthermore, DVNDA consistently outperforms Greedy at high dynamic rates, whereas other NCO methods experience performance degradation in these scenarios.

In summary, DVNDA not only provides better solution quality than existing NCO methods, but it does so with similar computational efficiency and robust performance even in chal-

lenging high-dynamic-rate conditions. Although existing NCO methods perform well in low dynamic rate environments, in complex scenarios with high dynamic rates and vehicle collaboration, the expert knowledge-based and centralized network architecture is prone to leading to biased vehicle assignment decisions, which in turn affects the overall performance. On the contrary, through parallel decision-making and efficient collaboration, DVNDA achieves high robustness and decision-making efficiency in dynamic environments.

C. Performance on Real-world Instances

To evaluate the performance of DVNDA in a real-world environment, we adopted the street network data of Vienna (Austria), which was obtained from OpenStreetMap. First, we acquired all the node and road data of the city of Vienna [42]. The resulting graph contains 16,080 nodes (intersections) and 36,424 arcs (street segments), as shown in Fig. 5. We normalized the coordinates of all nodes to the range $[0, 1]$. Based on the normalized data, we randomly selected instances at scales of 20, 35, and 50, with 100 instances at each scale used as the test set. Parameters such as node dynamics and the number of vehicles were randomly generated according to the settings in this paper.

The results on this real-world data are shown in Table II. DVNDA significantly outperforms all baseline methods across different scales and dynamic rates. For instance, at $\varphi = 75\%$ and $n = 50$, DVNDA achieves a cost of 17.71 with 100% QoS, while the best competing method, AMCVN, has a cost of 17.96 with 100% QoS. DVNDA consistently demonstrates superior cost optimization while maintaining competitive service quality, validating its practical applicability in real-world

TABLE II: Detailed comparisons between Greedy, MARDAM, MAAM, LiDRL, AMCVN, and DVNDA on real-world instances (100 instances). **Bold** means the best results.

φ	Method	$n = 20$			$n = 35$			$n = 50$		
		Cost	QoS	Time	Cost	QoS	Time	Cost	QoS	Time
10%	Greedy	5.57 $\pm$ 0.86	99.85%	—	9.18 $\pm$ 1.13	99.95%	—	12.70 $\pm$ 1.48	99.95%	—
	MARDAM	5.52 $\pm$ 0.69	99.80%	1s	9.15 $\pm$ 1.93	99.90%	1s	12.34 $\pm$ 1.27	99.95%	2s
	MAAM	5.41 $\pm$ 0.58	99.65%	1s	9.07 $\pm$ 1.85	99.95%	1s	12.07 $\pm$ 1.07	99.85%	2s
	LiDRL	5.34 $\pm$ 0.65	100%	1s	8.92 $\pm$ 1.82	100%	2s	11.92 $\pm$ 1.34	100%	3s
	AMCVN	5.27 $\pm$ 0.71	100%	1s	8.85 $\pm$ 1.92	100%	2s	11.85 $\pm$ 2.01	100%	3s
	DVNDA	5.02 $\pm$ 0.62	100%	1s	8.52 $\pm$ 1.01	100%	3s	11.60 $\pm$ 1.09	100%	4s
25%	Greedy	6.09 $\pm$ 0.98	99.85%	—	10.13 $\pm$ 1.30	100%	—	13.95 $\pm$ 1.52	99.95%	—
	MARDAM	6.13 $\pm$ 0.96	99.75%	1s	10.17 $\pm$ 1.39	99.75%	1s	13.97 $\pm$ 1.70	98.85%	3s
	MAAM	6.06 $\pm$ 1.13	99.90%	1s	10.09 $\pm$ 1.42	99.50%	1s	13.62 $\pm$ 1.85	99.50%	3s
	LiDRL	5.98 $\pm$ 1.05	100%	1s	9.96 $\pm$ 1.38	100%	2s	13.46 $\pm$ 1.44	100%	4s
	AMCVN	5.82 $\pm$ 0.87	100%	1s	9.86 $\pm$ 1.42	100%	2s	13.25 $\pm$ 1.47	100%	4s
	DVNDA	5.52 $\pm$ 0.78	100%	1s	9.64 $\pm$ 1.19	100%	3s	12.85 $\pm$ 1.38	100%	5s
50%	Greedy	7.11 $\pm$ 1.02	99.90%	—	12.03 $\pm$ 1.47	99.95%	—	16.50 $\pm$ 1.71	99.95%	—
	MARDAM	7.23 $\pm$ 1.08	99.95%	1s	12.22 $\pm$ 1.19	99.65%	1s	17.12 $\pm$ 1.74	99.55%	3s
	MAAM	7.15 $\pm$ 1.24	99.85%	1s	12.08 $\pm$ 1.22	99.55%	1s	16.48 $\pm$ 1.55	99.80%	3s
	LiDRL	6.98 $\pm$ 1.13	100%	1s	11.98 $\pm$ 1.12	100%	2s	16.09 $\pm$ 1.59	100%	4s
	AMCVN	6.81 $\pm$ 0.96	100%	1s	11.83 $\pm$ 1.31	100%	2s	15.73 $\pm$ 1.43	100%	4s
	DVNDA	6.41 $\pm$ 0.87	100%	1s	11.72 $\pm$ 1.32	100%	3s	15.18 $\pm$ 1.57	100%	5s
75%	Greedy	7.77 $\pm$ 0.88	99.90%	—	13.37 $\pm$ 1.43	99.85%	—	18.90 $\pm$ 1.69	99.90%	—
	MARDAM	7.93 $\pm$ 1.01	99.95%	1s	13.74 $\pm$ 1.53	99.15%	1s	19.84 $\pm$ 2.05	99.60%	3s
	MAAM	7.85 $\pm$ 1.05	99.90%	1s	13.53 $\pm$ 1.49	99.60%	1s	19.13 $\pm$ 1.72	99.50%	3s
	LiDRL	7.79 $\pm$ 0.95	100%	1s	13.38 $\pm$ 1.17	100%	2s	18.37 $\pm$ 1.91	100%	4s
	AMCVN	7.54 $\pm$ 0.98	100%	1s	13.26 $\pm$ 1.46	100%	2s	17.96 $\pm$ 1.82	100%	4s
	DVNDA	7.21 $\pm$ 0.94	100%	1s	13.07 $\pm$ 1.66	100%	3s	17.71 $\pm$ 1.55	100%	5s

TABLE III: Detailed comparisons between MARDAM, MAAM, LiDRL, AMCVN, and DVNDA on cross-scale instances (400 instances). **Bold** means the best results.

Method	$n = 60$			$n = 70$			$n = 80$		
	Cost	QoS	Time	Cost	QoS	Time	Cost	QoS	Time
MARDAM	33.93 $\pm$ 6.01	98.25%	10s	48.74 $\pm$ 15.53	95.15%	11s	80.84 $\pm$ 25.05	83.60%	13s
MAAM	33.25 $\pm$ 7.05	98.90%	10s	47.53 $\pm$ 15.49	95.60%	11s	78.13 $\pm$ 27.72	82.50%	13s
LiDRL	32.86 $\pm$ 6.95	99.80%	14s	45.93 $\pm$ 17.87	96.00%	16s	76.37 $\pm$ 26.91	86.40%	17s
AMCVN	31.54 $\pm$ 6.98	99.70%	14s	43.60 $\pm$ 19.15	96.30%	16s	73.96 $\pm$ 26.82	85.65%	17s
DVNDA	28.80 $\pm$ 6.70	99.80%	16s	40.46 $\pm$ 15.63	97.70%	17s	69.52 $\pm$ 26.59	91.10%	19s

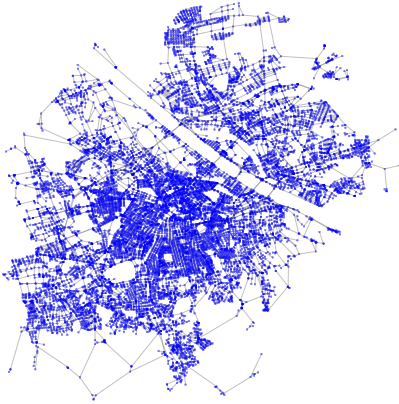


Fig. 5: Graph representing the street network of Vienna, Austria.

superior performance across various problem scales, dynamic rates, and real-world conditions, making it a robust solution for DCVRP.

D. Generalization

1) *Performance on cross-scale instances:* To further examine the scalability and robustness of the proposed method, we evaluate its generalization ability by testing on larger problem sizes ($n = 60, 70, 80$) beyond the training scale. The results in Table III are obtained by averaging over four dynamic rates (0.1, 0.25, 0.5, 0.75), using a model trained with $n = 50$ customers and generalized to larger scales with $n = 60, 70, 80$, while fixing the fleet size at $m = 10$. The comparison shows that DVNDA consistently achieves lower routing costs while maintaining high QoS across different scales. In particular, QoS remains stable without significant degradation, indicating that the method can serve more customers effectively even as the problem size grows. The inference time is comparable to other NCO methods, ensuring practicality in real-time decision-making. It should also be noted that the variance in

scenarios. The experimental results confirm that DVNDA's distributed architecture and collaborative mechanisms enable

routing cost increases under different dynamic rates, which explains the relatively larger deviations in the reported results.

Overall, when tackling large-scale DCVRP under high dynamic rates, the centralized method with parameter sharing tends to suffer from decision bias. By contrast, DVNDA enables parallel decision-making through independent neural networks for each vehicle, fostering efficient collaboration and leading to superior service quality and reduced costs.

2) *Performance on cross-distribution instances*: To assess generalization to different customer spatial distributions, we tested all methods on two distinct distributions (Clustered and Mixed) that were not necessarily seen during training. Table IV summarizes the outcomes for 20-customer instances under various dynamic rates. DVNDA exhibits excellent cross-distribution generalization, delivering the best performance under both clustered and mixed customer layouts. In all dynamic settings, DVNDA manages to keep its route cost low and QoS high regardless of how the customers are geographically distributed. At the highest dynamic rate ($\varphi = 75\%$), for example, DVNDA achieves the lowest total cost in both the cluster scenario and the mixed scenario, while also managing to serve all customers in our 20-scale tests (100% QoS). In contrast, some baseline methods struggle when faced with a distribution shift. Their costs are higher, and their QoS can drop (for instance, MARDAM and MAAM see noticeable declines in QoS under these unseen distributions). The fact that DVNDA performs robustly on both clustered and mixed instances indicates that its policy is not over-specialized to a particular spatial pattern. Instead, by leveraging distributed decision-making, DVNDA can flexibly adapt to different demand distributions, maintaining efficient routing and service reliability even when the customer distribution differs from the training conditions.

3) *Performance on different numbers of vehicles*: To demonstrate the generalization ability of our method, we conducted an additional experiment by transferring a pretrained model to a new vehicle setting. Specifically, we first trained a model on the setting with 50 customers and 10 vehicles. Then, when adapting to the 12-vehicle scenario, we kept the parameters of the first 10 vehicle networks unchanged, while randomly initializing the parameters of the additional two vehicle networks (the 11th and 12th). With only a small amount of additional training (5 epochs), the new model was able to quickly learn effective strategies for the added vehicles.

Fig. 6 results show that DVNDA maintains strong performance after adaptation, consistently achieving the lowest path cost under all dynamic rates (0.1, 0.25, 0.5, 0.75). This indicates that although the setting changes with more vehicles, the pretrained knowledge transfers well, and only limited fine-tuning is needed for the new networks to align with the existing strategy. It should be noted that when the number of vehicles increased from 10 to 12, the total cost of the path increased slightly for all methods. This may be due to additional vehicles creating redundant routes without optimal coordination, leading to a longer total travel distance. In general, DVNDA's distributed decision making and aggregation help vehicles collaborate more effectively, delivering the best performance in different fleet sizes and levels of dynamism.

TABLE IV: Detailed comparisons between Greedy, MARDAM, MAAM, LiDRL, AMCVN, and DVNDA on two different distributions of 20-scale (100 instances). **Bold** means the best results.

φ	Method	Cluster			Mixed (cluster and uniform)		
		Cost	QoS	Time	Cost	QoS	Time
10%	Greedy	11.05 $\pm$ 1.57	99.85%	—	9.18 $\pm$ 1.14	99.90%	—
	MARDAM	10.46 $\pm$ 1.60	99.85%	1s	9.15 $\pm$ 1.23	99.85%	1s
	MAAM	10.09 $\pm$ 1.73	99.80%	1s	9.07 $\pm$ 1.22	99.80%	1s
	LiDRL	9.76 $\pm$ 1.86	100%	1s	9.01 $\pm$ 1.19	100%	1s
	AMCVN	9.44 $\pm$ 1.68	100%	1s	8.95 $\pm$ 1.15	100%	2s
	DVNDA	9.04 $\pm$ 1.70	100%	1s	7.88 $\pm$ 1.10	100%	2s
25%	Greedy	11.33 $\pm$ 1.65	99.85%	—	9.83 $\pm$ 1.41	99.95%	—
	MARDAM	11.35 $\pm$ 1.80	99.80%	1s	9.96 $\pm$ 1.36	99.90%	1s
	MAAM	10.92 $\pm$ 1.76	98.90%	1s	9.59 $\pm$ 1.45	99.65%	1s
	LiDRL	10.58 $\pm$ 1.69	100%	1s	9.25 $\pm$ 1.22	100%	2s
	AMCVN	10.07 $\pm$ 1.73	100%	1s	8.80 $\pm$ 1.37	100%	2s
	DVNDA	9.52 $\pm$ 1.72	100%	2s	8.58 $\pm$ 1.17	100%	2s
50%	Greedy	12.41 $\pm$ 1.80	99.90%	—	11.14 $\pm$ 1.36	99.65%	—
	MARDAM	12.74 $\pm$ 1.44	99.60%	1s	11.31 $\pm$ 1.71	99.75%	1s
	MAAM	12.18 $\pm$ 1.61	98.75%	1s	10.96 $\pm$ 1.47	99.30%	1s
	LiDRL	11.44 $\pm$ 1.83	100%	1s	10.70 $\pm$ 1.65	100%	2s
	AMCVN	10.53 $\pm$ 1.93	100%	1s	10.41 $\pm$ 1.54	100%	2s
	DVNDA	9.99 $\pm$ 1.71	100%	2s	9.97 $\pm$ 1.36	100%	2s
75%	Greedy	13.35 $\pm$ 2.23	99.50%	—	12.15 $\pm$ 1.36	99.70%	—
	MARDAM	13.86 $\pm$ 1.85	98.10%	1s	12.75 $\pm$ 1.42	98.70%	1s
	MAAM	13.43 $\pm$ 1.72	98.70%	1s	12.44 $\pm$ 1.54	98.40%	1s
	LiDRL	12.25 $\pm$ 1.84	100%	2s	12.18 $\pm$ 1.63	100%	2s
	AMCVN	11.08 $\pm$ 1.54	100%	2s	11.75 $\pm$ 1.47	100%	2s
	DVNDA	10.18 $\pm$ 1.64	100%	2s	11.38 $\pm$ 1.49	100%	2s

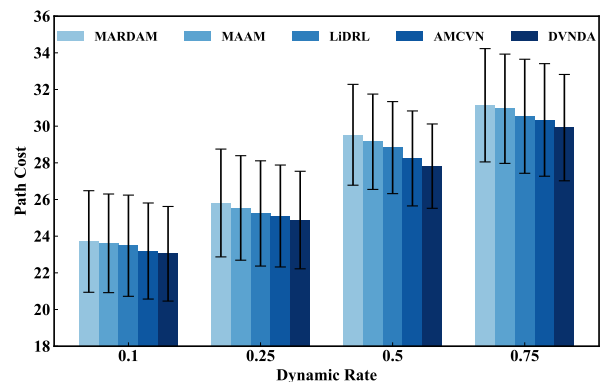


Fig. 6: Path cost comparison under varying dynamic rates for 50 customers and 12 vehicles.

4) *Performance under extreme dynamics*: To further examine the applicability boundary of DVNDA, we evaluate all methods under an extreme dynamic rate of $\varphi = 90\%$ on 20-customer instances, where almost all customers arrive unexpectedly during service. The results are summarized in Table V. Under $\varphi = 90\%$, all methods are tested in a near-fully dynamic environment. Greedy remains cost-competitive, but its QoS slightly decreases to 98.90%, while MAAM also shows a minor QoS reduction. By contrast, DVNDA continues to achieve 100% QoS and attains the lowest cost of 12.49 among the non-dedicated baselines. In addition, the cost increase from $\varphi = 75\%$ to 90% is more pronounced than that from 50% to 75%, indicating that the problem difficulty

accelerates as the system approaches full dynamism.

TABLE V: Performance under extreme dynamic rate on 20-scale instances (100 instances). **Bold** means the best results.

φ	Method	Cost	QoS	Time
90%	Greedy	12.68 ± 1.88	98.90%	—
	MARDAM	13.15 ± 1.96	99.50%	1s
	MAAM	13.11 ± 1.92	99.10%	1s
	LiDRL	12.97 ± 1.90	100%	1s
	AMCVN	12.62 ± 1.83	100%	1s
	DVNDA	12.49 ± 1.89	100%	1s

E. Sensitivity Analysis

1) *Impact of the penalty coefficient α* : To evaluate the impact of the penalty coefficient α on the performance of DVNDA, we conducted a sensitivity analysis at 20-scale instances and tested the performance when α took the values of 2, 3, 4, 5, and 6, with all dynamic rates (0.1, 0.25, 0.5, 0.75). The experimental results are shown in Table VI. It can be seen that DVNDA achieves near-optimal cost and quality of service at $\alpha = 5$ under all dynamic rates, provides the best performance balance at different dynamic rates, and is ideal for DVNDA.

TABLE VI: Sensitivity analysis of DVNDA on 20-scale instances (100 instances). **Bold** means the best results methods.

φ	α	$n = 20$		
		Cost	QoS	Time
10%	2	8.37 ± 1.23	100%	1s
	3	8.39 ± 1.29	100%	1s
	4	8.34 ± 1.21	100%	1s
	5	8.31 ± 1.22	100%	1s
	6	8.38 ± 1.25	100%	1s
25%	2	9.09 ± 1.37	100%	1s
	3	8.96 ± 1.34	100%	1s
	4	9.08 ± 1.32	100%	1s
	5	8.95 ± 1.30	100%	1s
	6	9.07 ± 1.39	100%	1s
50%	2	10.58 ± 1.56	100%	1s
	3	10.55 ± 1.52	100%	1s
	4	10.51 ± 1.49	100%	1s
	5	10.47 ± 1.53	100%	1s
	6	10.54 ± 1.52	100%	1s
75%	2	11.96 ± 1.42	100%	1s
	3	11.88 ± 1.48	100%	1s
	4	11.84 ± 1.59	100%	1s
	5	11.78 ± 1.44	100%	1s
	6	11.85 ± 1.40	100%	1s

2) *Impact of intervals β* : To assess the sensitivity of the interval number β , we train separate DVNDA models with $\beta \in \{6, 8, 10, 12, 14\}$ on 20-customer instances for each dynamic rate independently, and report both the path cost and training time per epoch. The results are shown in Fig. 7. At low dynamic rates ($\varphi = 10\%$), the path cost remains virtually constant across all β values, since most customers are known at the outset and the replanning frequency has negligible impact. In this regime, increasing β only raises training time with no routing benefit. As the dynamic rate

grows, β becomes progressively more influential. At $\varphi = 75\%$, a smaller β reduces re-encoding overhead and accelerates training, but provides fewer replanning opportunities, weakening the system's responsiveness to late-arriving customers and resulting in a higher cost of 11.97. Increasing β to 10 enables the model to react more promptly to dynamically revealed requests, reducing the cost to 11.78. However, further increasing β beyond 10 yields diminishing returns. The cost at $\beta = 12$ and $\beta = 14$ is nearly identical to that at $\beta = 10$, while training time continues to grow from 42 to 52 min/epoch. The same plateau pattern is consistently observed at $\varphi = 25\%$ and 50% . Therefore, $\beta = 10$ is selected as the default, providing the best trade-off between dynamic responsiveness and computational efficiency across all tested dynamic rates.

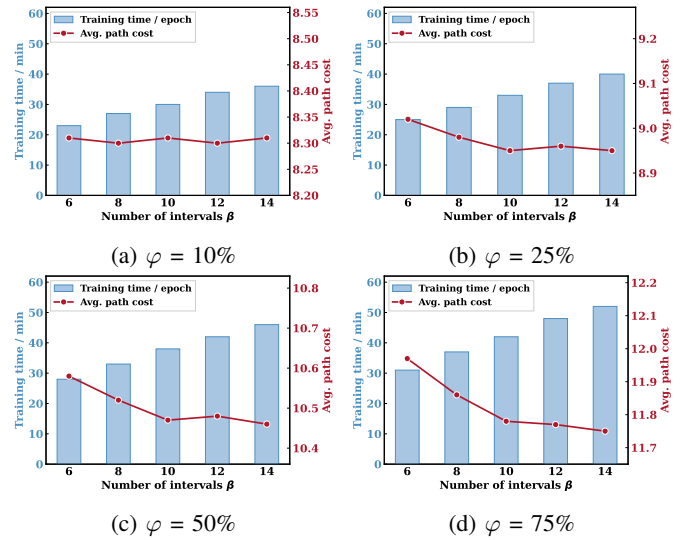


Fig. 7: Sensitivity analysis of the number of intervals β on 20-customer instances under different dynamic rates.

V. CONCLUSION

This paper aims to improve the performance of the NCO method for handling DCVRP in collaborative vehicle decision-making. This objective is successfully achieved by proposing DVNDA. DVNDA enables each vehicle to make independent and collaborative decisions based on the shared status of all vehicles and current customers. Extensive experimental results demonstrated that DVNDA significantly outperformed both the greedy algorithm and existing NCO methods across various problem scales and degrees of dynamism, achieving lower cost and higher QoS.

Despite the promising results, DVNDA still has room for improvement. First, the interval update requires re-encoding, which may introduce computational overhead. Future research should explore lightweight update strategies for handling dynamic problems more efficiently. Second, the current model considers only vehicle capacity constraints. Extending the framework to incorporate time window constraints is a natural next step. This would require modifying the feasibility masks and the reward function to penalize time window violations, but the core distributed architecture would remain applicable.

Third, incorporating heterogeneous vehicle fleets (with varying capacities, speeds, or service capabilities) would require extending the vehicle state representation and potentially learning fleet-type-specific network parameters, though DVNDA's independent-network-per-vehicle design is inherently compatible with such heterogeneity.

REFERENCES

- [1] G. D. Konstantakopoulos, S. P. Gayialis, and E. P. Kechagias, "Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification," *Operational research*, pp. 1–30, 2020.
- [2] J. Wang, Y. Sun, Z. Zhang, and S. Gao, "Solving multitrip pickup and delivery problem with time windows and manpower planning using multiobjective algorithms," *IEEE/CAA Journal of Automatica Sinica*, vol. 7, no. 4, pp. 1134–1153, 2020.
- [3] A. Mor and M. G. Speranza, "Vehicle routing problems over time: a survey," *Annals of Operations Research*, vol. 314, no. 1, pp. 255–275, 2022.
- [4] B. H. O. Rios, E. C. Xavier, F. K. Miyazawa, P. Amorim, E. Curcio, and M. J. Santos, "Recent dynamic vehicle routing problems: A survey," *Computers & Industrial Engineering*, vol. 160, p. 107604, 2021.
- [5] Y.-H. Jia, W.-N. Chen, T. Gu, H. Zhang, H. Yuan, Y. Lin, W.-J. Yu, and J. Zhang, "A dynamic logistic dispatching system with set-based particle swarm optimization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 48, no. 9, pp. 1607–1621, 2017.
- [6] J. Zhang and T. Van Woensel, "Dynamic vehicle routing with random requests: A literature review," *International Journal of Production Economics*, vol. 256, p. 108751, 2023.
- [7] A. Attanasio, J.-F. Cordeau, G. Ghiani, and G. Laporte, "Parallel tabu search heuristics for the dynamic multi-vehicle dial-a-ride problem," *Parallel Computing*, vol. 30, no. 3, pp. 377–387, 2004.
- [8] J. Euchi, A. Yassine, and H. Chabchoub, "The dynamic vehicle routing problem: Solution with hybrid metaheuristic approach," *Swarm and Evolutionary Computation*, vol. 21, pp. 41–53, 2015.
- [9] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.
- [10] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" *arXiv preprint arXiv:1803.08475*, 2018.
- [11] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "Pomo: Policy optimization with multiple optima for reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 188–21 198, 2020.
- [12] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: a methodological tour d'hORIZON," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.
- [13] S. Liu, Y. Zhang, K. Tang, and X. Yao, "How good is neural combinatorial optimization? a systematic evaluation on the traveling salesman problem," *IEEE Computational Intelligence Magazine*, vol. 18, no. 3, pp. 14–28, 2023.
- [14] C. Wang, Z. Yu, S. McAleer, T. Yu, and Y. Yang, "Asp: Learn a universal neural solver!" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 6, pp. 4102–4114, 2024.
- [15] Y. Wang, Y.-H. Jia, W.-N. Chen, and Y. Mei, "Distance-aware attention reshaping for enhancing generalization of neural solvers," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 18 900–18 914, 2025.
- [16] Y. Wu, W. Song, Z. Cao, J. Zhang, and A. Lim, "Learning improvement heuristics for solving routing problems," *IEEE transactions on neural networks and learning systems*, vol. 33, no. 9, pp. 5057–5069, 2021.
- [17] Z. Zhang, H. Liu, M. Zhou, and J. Wang, "Solving dynamic traveling salesman problems with deep reinforcement learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 4, pp. 2119–2132, 2021.
- [18] W. Pan and S. Q. Liu, "Deep reinforcement learning for the dynamic and uncertain vehicle routing problem," *Applied Intelligence*, vol. 53, no. 1, pp. 405–422, 2023.
- [19] T. Mustakhov, Y. Akhmetbek, and A. Bogyrbayeva, "Deep reinforcement learning for stochastic dynamic vehicle routing problem," in *2023 17th International Conference on Electronics Computer and Computation (ICECCO)*. IEEE, 2023, pp. 1–5.
- [20] R. Gama, D. Fuertes, C. R. del Blanco, and H. L. Fernandes, "Multi-agent environments for vehicle routing problems," *arXiv preprint arXiv:2411.14411*, 2024.
- [21] J. James, W. Yu, and J. Gu, "Online vehicle routing with neural combinatorial optimization and deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3806–3817, 2019.
- [22] W. Joe and H. C. Lau, "Deep reinforcement learning approach to solve dynamic vehicle routing problem with stochastic customers," in *Proceedings of the international conference on automated planning and scheduling*, vol. 30, 2020, pp. 394–402.
- [23] G. Bono, J. S. Dibangoye, O. Simonin, L. Matignon, and F. Pereyron, "Solving multi-agent routing problems using deep attention mechanisms," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 12, pp. 7804–7813, 2020.
- [24] K. Zhang, F. He, Z. Zhang, X. Lin, and M. Li, "Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach," *Transportation Research Part C: Emerging Technologies*, vol. 121, p. 102861, 2020.
- [25] J. Li, Y. Ma, R. Gao, Z. Cao, A. Lim, W. Song, and J. Zhang, "Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem," *IEEE Transactions on Cybernetics*, vol. 52, no. 12, pp. 13 572–13 585, 2022.
- [26] C. Xiang, Z. Wu, J. Tu, and J. Huang, "Centralized deep reinforcement learning method for dynamic multi-vehicle pickup and delivery problem with crowdshippers," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 8, pp. 9253–9267, 2024.
- [27] A. Haghani and S. Jung, "A dynamic vehicle routing problem with time-dependent travel times," *Computers & operations research*, vol. 32, no. 11, pp. 2959–2986, 2005.
- [28] V. Pillac, M. Gendreau, C. Gu  ret, and A. L. Medaglia, "A review of dynamic vehicle routing problems," *European Journal of Operational Research*, vol. 225, no. 1, pp. 1–11, 2013.
- [29] B. Li, G. Wu, Y. He, M. Fan, and W. Pedrycz, "An overview and experimental study of learning-based optimization algorithms for the vehicle routing problem," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 7, pp. 1115–1138, 2022.
- [30] J. Li, G. Wu, M. Fan, Z. Cao, and Y. Wang, "Heterogeneous attention-based graph convolutional network for solving asymmetric pickup and delivery problem," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 13 000–13 014, 2025.
- [31] C. Wang, Z. Cao, Y. Wu, L. Teng, and G. Wu, "Deep reinforcement learning for solving vehicle routing problems with backhauls," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 3, pp. 4779–4793, 2024.
- [32] K. Lin, R. Zhao, Z. Xu, and J. Zhou, "Efficient large-scale fleet management via multi-agent deep reinforcement learning," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 1774–1783.
- [33] Z. Zong, M. Zheng, Y. Li, and D. Jin, "Mapdp: Cooperative multi-agent reinforcement learning to solve pickup and delivery problems," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, 2022, pp. 9980–9988.
- [34] X. Zhang, Y. Yang, J. Cai, Q. Zhu, W. Chen, and Q. Lin, "Deep reinforcement learning-based multi-agent algorithm for vehicle routing problem in complex logistics scenarios," in *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024, pp. 1–8.
- [35] C. Hua, F. Berto, J. Son, S. Kang, C. Kwon, and J. Park, "Camp: Collaborative attention model with profiles for vehicle routing problems," in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, 2025, pp. 1015–1024.
- [36] Z. Zhang, J. Yang, Z. Cao, and H. C. Lau, "Neuro-ins: A learning-based one-shot node insertion for dynamic routing problems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 9, pp. 5495–5507, 2025.
- [37] N. N. Sultana, V. Baniwal, A. Basumatary, P. Mittal, S. Ghosh, and H. Khadilkar, "Fast approximate solutions using reinforcement learning for dynamic capacitated vehicle routing with time windows," *arXiv preprint arXiv:2102.12088*, 2021.
- [38] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, pp. 229–256, 1992.
- [39] S. Lin, H. Cui, Y. Wang, and Y.-H. Jia, "Decoupled training neural solver for dynamic traveling salesman problem," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 12 986–12 992.

- [40] G. Kim, Y. S. Ong, T. Cheong, and P. S. Tan, "Solving the dynamic vehicle routing problem under traffic congestion," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 8, pp. 2367–2380, 2016.
- [41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, pp. 6000–6010, 2017.
- [42] J. Zhang, K. Luo, A. M. Florio, and T. Van Woensel, "Solving large-scale dynamic vehicle routing problems with stochastic requests," *European Journal of Operational Research*, vol. 306, no. 2, pp. 596–614, 2023.